Decodificación de habla desde señales cerebrales: réplica causal de Willett et al. (2023)

Asier Aurre

1. Qué he hecho

He replicado en PyTorch el sistema de *A high-performance speech neuroprosthesis* (Willett et al., 2023), donde un participante con parálisis (T12) intenta decir frases mientras dos matrices de microelectrodos registran su corteza premotora; una red neuronal recurrente convierte esa actividad en probabilidades de fonemas y un modelo de lenguaje reconstruye las palabras. Mi réplica reproduce tanto las versiones online como offline, aunque la demo solo usa la última versión online (el modelo solo mira al pasado y nunca al futuro).

Métrica	Paper (online, vocalizado)	Mi réplica causal	Condiciones de mi evaluación
PER (error de fonemas)	19,7 %	31,90 %	1.460 frases de validación, por bloques de 80 ms
WER (error de palabras)	23,8 %	37,95 %	Mismas frases, con el trigrama de 125.000 palabras

Las cifras no son directamente comparables: utilizo menos sesiones de entrenamiento, una partición distinta y normalización fija por sesión.

2. Los datos y la partición

Parto de `competitionData`, publicado por los autores en Dryad. Como las descargas directas desde Dryad fallaban por sus bloqueos antirobot, subí los paquetes a Google Drive y preparé `scripts/get_data.py`, que los descarga con reanudación, comprueba el tamaño y el SHA antes de descomprimirlos.

Cada ensayo corresponde a una frase y contiene, en ventanas (*bins*) de 20 ms, 256 canales de dos variables que se complementan. Los *cruces de umbral* (`tx1`) cuentan cuántas veces el voltaje cae por debajo de un umbral negativo en esos 20 ms, como el disparo de una neurona produce un pulso brusco, este conteo entero aproxima cuántos disparos hubo cerca del electrodo. La *potencia de espigas* (`spikePow`) mide la energía continua en la banda de frecuencias de las espigas, recogiendo también actividad que no llega a cruzar el umbral. Siguiendo la implementación de referencia de los autores, descarto los 128 canales del área 44 y conservo los 128 del área 6v de cada variable, por lo que cada bin de 20 ms entra al pipeline como un vector de 256 características (128 de `tx1` seguidas de 128 de `spikePow`).

Diferencias con el paper en datos y partición. El estudio original entrena con 10.850 frases. En mis primeras pruebas de viabilidad valoré usar solo los primeros 5 días (∼2.000 frases, un recorte que el propio paper muestra como referencia), y finalmente fijé el protocolo del proyecto en **19 sesiones**: 6.640 frases en `train/*.mat` más 680 en `test/*.mat`.Para poder elegir el mejor checkpoint y comparar variantes sin tocar nunca las 680 frases de test, reservo como validación los últimos bloques completos de grabación de cada sesión (cerca del 20 %, 1.460 frases) y entreno con las 5.180 restantes. Cortar por bloques enteros evita mezclar frases de una misma tanda entre entrenamiento y validación.

3. El flujo completo y la arquitectura

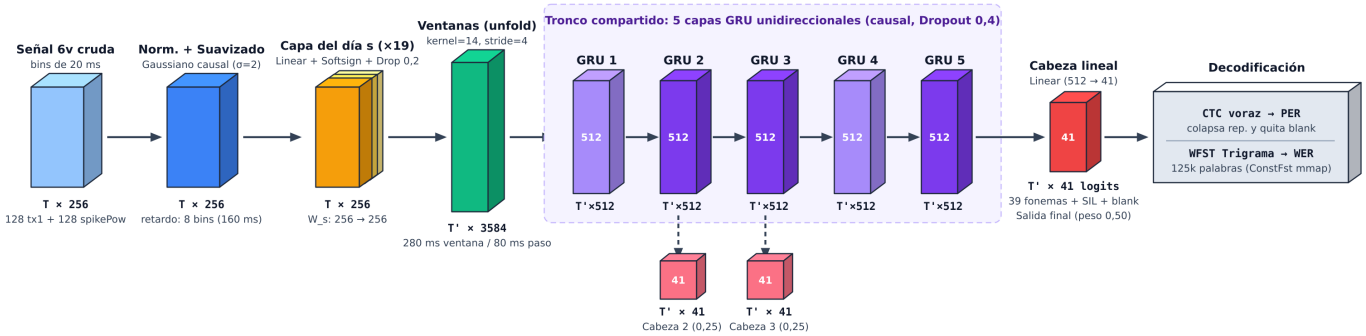
1. Etiquetas: del texto a fonemas. El sistema del paper no predice letras ni palabras completas directamente, porque la ortografía inglesa es muy irregular y con palabras sueltas habría muy pocos ejemplos de cada una; en su lugar predice los 39 fonemas del inglés (ARPAbet) más el símbolo de silencio `SIL` entre palabras y al final de la frase. Al igual que en el repositorio original, convierto cada transcripción con la librería `g2p-en` (basada en el diccionario CMUdict), que añade un dígito de *estrés* (acento prosódico) a cada vocal: por ejemplo, `you` produce `Y UW1` y `about` produce `AH0 B AW1 T`. Como el vocabulario de 40 símbolos no distingue acentos, el preprocesador limpia primero la puntuación, pasa a minúsculas y elimina esos dígitos, transformando `"You can see"` en `Y UW SIL K AE N SIL S IY SIL`. De cada ensayo conocemos esta secuencia objetivo, pero no en qué bin de 20 ms se pronuncia cada fonema.

2. Preprocesado de la señal. Sigue los tres pasos del paper, con dos adaptaciones propias para el modo causal y la demo:

- *Normalización por sesión.* Un canal puede moverse en valores de 1.000 y otro en torno a 5, y además, el nivel de un mismo electrodo cambia a lo largo de las horas de una misma sesión. Para igualar las escalas, a cada canal se le resta su media y se divide por su desviación típica. En el paper original reajustan estas estadísticas bloque a bloque para seguir esa deriva durante el día, yo, en cambio, calculo una única media y desviación fijas por sesión (usando exclusivamente train) y las aplico sin tocar validación ni test. Esto evita cualquier fuga de datos y simplifica el pipeline (aunque penaliza algo el PER en los últimos bloques del día), y al guardar esas 19 medias y desviaciones dentro de `mejor_val.pt`, la demo puede normalizar al instante cualquier grabación cruda sin tener que recalcular las estadísticas de entrenamiento al arrancar.
- *Ruido en entrenamiento.* Como el conjunto de frases es limitado para una red grande, el paper añade en cada paso de entrenamiento ruido blanco gaussiano (SD = 1,0) y un desplazamiento constante aleatorio por canal (SD = 0,2). Así el modelo ve una versión ligeramente distinta de la frase en cada época y se vuelve robusto a derivas de nivel. Nunca se aplica en validación ni en test.

- *Suavizado gaussiano causal.* La señal bin a bin es muy ruidosa, especialmente los conteos enteros de `tx1`. El paper aplica una convolución gaussiana temporal ($\sigma = 2{,}0$ bins) canal por canal. En su baseline offline el filtro es centrado y utiliza bins futuros, para que mi modelo y la demo sean estrictamente causales, desplazo el centro del filtro 8 bins (160 ms) hacia el pasado y conservo una cola de historial entre bloques.

3. Arquitectura de la red (`SpeechGRU`). He trasladado la arquitectura de TensorFlow/Keras a PyTorch manteniendo la inicialización original (identidad en las capas de día, Glorot en proyecciones y ortogonal en pesos recurrentes):



1. *Capa lineal por día.* Aunque se normalice cada sesión, los microelectrodos sufren pequeños desplazamientos entre días y el mismo canal no recoge exactamente las mismas neuronas en abril y en agosto. El modelo original coloca una capa lineal específica para cada una de las 19 sesiones ($256 \to 256$, inicializada en la identidad, seguida de activación `Softsign`). A estas capas no se les define a mano ningún espacio común: como las GRU posteriores son compartidas por todas las sesiones, el descenso por gradiente fuerza a cada capa de día a proyectar su sesión hacia la representación que el tronco común sabe interpretar.
2. *Agrupación en ventanas.* Un solo bin de 20 ms contiene muy poca información para distinguir un fonema y haría las secuencias demasiado largas para la GRU. Por eso la red agrupa paquetes de 14 bins consecutivos (280 ms) avanzando de 4 en 4 (80 ms) mediante parches solapados, aplanando cada ventana en un vector de $14 \times 256 = 3{,}584$.
3. *Cinco capas GRU unidireccionales y variantes propias.* Para la parte comun , se usan 5 capas GRU que mantienen un estado oculto de 512 unidades paso a paso; al ser unidireccionales, cada ventana solo depende de las anteriores. Mientras que el baseline offline uso 1.024 unidades y ventanas de 32 bins. Además, como **experimento propio**, añadí cabezas de salida tras las capas 2 y 3 (entrenadas con una pérdida CTC conjunta con pesos 0,25/0,25/0,50) y entrené redes independientes de 2 y 3 capas como control, para estudiar cuánta latencia se ahorra en CPU al cortar capas y cuánto PER cuesta; la demo final utiliza las 5 capas.
4. *Proyección a 41 clases.* Una capa lineal convierte las 512 unidades en 41 *logits* por cada paso de 80 ms: los 39 fonemas, `SIL` y el símbolo *blank*.

4. Entrenamiento con CTC. Como una frase de 500 bins produce unos 122 pasos de salida para unos 40 fonemas y no existe alineación temporal entre ambos, el paper entrena la red con la pérdida **CTC** (*Connectionist Temporal Classification*). Mediante programación dinámica, CTC suma la probabilidad de todos los caminos de longitud T' que, tras colapsar símbolos repetidos consecutivos y borrar el símbolo *blank* (`_`), resultan en la secuencia de fonemas correcta (por ejemplo, tanto `_ K K _ AE AE N _` como `K _ _ AE N _ _ _` colapsan en `K AE N`). El *blank* representa "no emitir fonema nuevo en este paso" y es necesario para separar dos fonemas idénticos seguidos, como `L _ L`. Siguiendo a los autores, optimizo la suma de pérdidas sin normalizar por longitud con Adam ($\epsilon = 0{,}1$, decaimiento lineal del *learning rate* de 0,02 a 0 en 10.000 pasos y regularización L2 de 10^{-5}). Como adaptación propia para no agotar la VRAM, divido cada lote de 64 frases en **4 micro-lotes de 16** y acumulo sus gradientes antes de actualizar los pesos.

5. Decodificación y modelo de lenguaje. Para medir el PER tomo en cada paso el símbolo más probable (*argmax* voraz), colapso repeticiones y retiro el *blank* (en streaming lo hace `IncrementalCTC` recordando el último símbolo entre bloques). Para obtener palabras y medir el WER conecto los logits de la GRU al decodificador WFST con el **trigrama original de 125.000 palabras** del paper, que combina las probabilidades acústicas con las probabilidades de secuencias de palabras. Prescindí del 5-grama y del rescoring con OPT-6.7B porque exigen demasiada memoria para correr en la CPU local y porque el 23,8 % online del paper con el que comparo tampoco utiliza OPT.

6. Aplicativo local (visor y llamadas). El servidor `scripts/demo_speech_http.py` levanta en local un visor web que transmite en vivo una grabación de validación en bloques de 4 bins (80 ms), Además añadí una segunda interfaz (`/llamada`) inspirada en demos recientes de neuralink: conecta con LiveKit, detecta cuando alguien habla por el micrófono, emula una respuesta eligiendo una muestra random del dataset , simula la llegada en tiempo real y sintetiza en voz las palabras a medida que se van estabilizando, usando Flite y FFmpeg.

4. Hardware, evolución del entrenamiento y reparto GPU / CPU

- Empecé midiendo tiempos e intentando entrenar en mi PC con una RTX 2060. Al lanzar lotes de 64 frases tuve primero un error de memoria CUDA, que resolví implementando la acumulación de gradientes en micro-lotes de 16. Aun así, mi GPU tenía problemas y se me bloqueaba el ordenador tras unas pocas épocas, por lo que empece a empaquetar el pipeline para ejecutarlo en Google Colab. En Colab gratuito con una Tesla T4 pude completar pruebas y la primera ronda de comparativas, pero los entrenamientos largos de 10.001 pasos se cortaban por límites de uso,

(como el run `m6sn4um1`, cortado en el paso 5.801). Por ello implementé primero el guardado completo del estado de entrenamiento (`-reanudar`, que restaura modelo, Adam, paso, RNG y el mismo run de W&B) pero finalmente acabe usando Colab Pro para poder entrenar el modelo causal en una **NVIDIA A100 de 40 GB**, completando los 10.001 pasos con todas sus validaciones en **34 min 52 s**

- **Compactación del trigrama para que quepa en RAM local.** El grafo WFST del trigrama original (`TLG.fst`, formato `VectorFst`) necesita muchísima RAM al cargarse entero en memoria, lo que hacía inviable abrirlo en mi ordenador (con 24GB). Para solucionarlo sin hacer más pequeño el vocabulario, creé `scripts/compact_speech_trigram.py`: en una instancia de Colab con RAM alta convierto una sola vez el grafo a formato `ConstFst` alineado de OpenFst 1.6.5 y luego, en mi ordenador, lo abro en modo **mapeado en memoria (`mmap`)**. Aunque el archivo ocupa mucho más en disco, el sistema operativo solo carga en RAM las páginas que la búsqueda consulta en cada frase, dejando el pico conjunto de GRU + trigrama en solo **3,30 GiB de RAM**

Tarea	Dónde se ejecuta	Motivo y configuración
Entrenamiento (10.001 pasos) y evaluación masiva en val	**GPU en Colab** (T4 / A100 40 GB)	Cómputo intensivo de las 5 GRU y CTC sobre 5.180 frases; notebooks 07–12 conectados a Weights & Biases (`tiiac-2026`).
Conversión del trigrama a `ConstFst` y evaluación WER completa	**CPU en Colab** (alta RAM)	Convertir el grafo original exige mucha memoria libre; una vez generado el `ConstFst` se descarga a la máquina local (notebook 13).
Inferencia de la demo HTTP, llamadas y pruebas (`-smoke`)	**CPU local** (2 hilos, sin CUDA)	La demo corre 100 % en CPU local con el trigrama mapeado ($\leq$ 3,30 GiB RSS).

Variante (10.001 pasos, seed 0)	Modo (H, ventana)	Salida	Paso	PER val	WER val	Latencia p50
Baseline 5 capas (`ofxo8x9a`)	Offline (1024, 32)	Capa 5	9.751	**27,69 %**	—	6,13 ms/frase (fwd 5,08 ms)
CTC conjunta 2/3/5 (`7b2n7v3o`)	Offline (1024, 32)	Capa 2	7.651	29,53 %	—	**3,53 ms/frase** (fwd 2,54 ms)
CTC conjunta 2/3/5 (`7b2n7v3o`)	Offline (1024, 32)	Capa 3	7.651	28,89 %	—	4,43 ms/frase (fwd 3,39 ms)
CTC conjunta 2/3/5 (`7b2n7v3o`)	Offline (1024, 32)	Capa 5	7.651	28,10 %	—	6,17 ms/frase (fwd 5,09 ms)
Control 2 capas (`153amw3i`)	Offline (1024, 32)	Capa 2	9.601	29,60 %	—	3,57 ms/frase (fwd 2,54 ms)
Control 3 capas (`yzcwdxn1`)	Offline (1024, 32)	Capa 3	9.301	27,89 %	—	4,34 ms/frase (fwd 3,29 ms)
Causal online (`jaovk23p`)	**Causal** (512, 14)	**Capa 5**	**9.951**	**31,90 %**	**37,95 %**	2,02 ms/paq. (4,55 ms +WFST)